

■ NETWORK SCANNING MASTERCLASS

A COMPLETE HACKER'S GUIDE TO RECON, PORT
SCANNING & VULNERABILITY DISCOVERY

■■■ Nitish Mukhiya

Security Researcher | Bug Bounty Hunter |
Ethical Hacker

LinkedIn: [linkedin.com/in/nitishmukhiya/](https://www.linkedin.com/in/nitishmukhiya/)

X (Twitter): [x.com/NITISHMUKHIYAJ](https://twitter.com/NITISHMUKHIYAJ)

GitHub: github.com/nitishmukhiyaji

Medium: medium.com/@nitishmukhiya

[CYBERSECURITY • BUG BOUNTY • ETHICAL
HACKING]

■ TABLE OF CONTENTS

01. ASN Enumeration

- ARIN & BGP.he.net
- ASNMAP Tool
- CIDR Collection

02. IP Enumeration

- MAPCIDR Tool
- IP Extraction
- CIDR Expansion

03. Host Discovery

- DNSX Reverse PTR
- Hostname Resolution
- Live Host Detection

04. Port Scanning

- Naabu
- Rustscan
- Masscan
- Angry IP Scanner

05. Nmap Complete Guide

- Basic/Advanced
- NSE Scripts
- Scan Types
- Output Formats

06. WAF & Firewall Evasion

- Fragmentation
- Decoys
- Slow Scans
- Source Port

07. Bug Bounty Workflows

- Quick Scan
- Full Port Scan
- Mass Scan
- Elite Pipeline

08. High Value Ports

- Port Reference Table

- Attack Surface
- Bug Bounty Value

09. Host Discovery Techniques

- ARP
- ICMP
- TCP SYN
- TCP ACK
- UDP

10. Shodan Recon

- Hostname Search
- IP Whois
- Filter Techniques

11. Sandmap

- Installation
- Commands
- Usage Guide

12. Scancannon

- Setup
- CIDR Scanning
- Usage

13. Cheat Sheet

- All Commands Quick Reference

14. Mind Map

- Network Scanning Master Map

■ STEP-BY-STEP LEARNING PATH

■ Phase 1: ASN Enumeration

Find the target organization's ASN numbers using ARIN and BGP.he.net lookup portals.

■ Phase 2: CIDR Collection

Use ASNMAP to extract all CIDR ranges associated with the discovered ASN numbers.

■ Phase 3: IP Extraction

Expand all CIDR ranges into individual IP addresses using MAPCIDR tool.

■ Phase 4: Hostname Discovery

Use DNSX with reverse PTR lookups to convert IPs to hostnames/domains.

■ Phase 5: Live Host Detection

Identify which hosts are alive using nmap -sn, masscan, or angry IP scanner.

■ Phase 6: Port Scanning

Scan all live hosts for open ports using Naabu, Masscan, or Rustscan.

■ Phase 7: Service Enumeration

Run nmap -sV on discovered open ports to identify running services and versions.

■ Phase 8: Deep Nmap Scanning

Execute NSE scripts and OS detection on interesting services found.

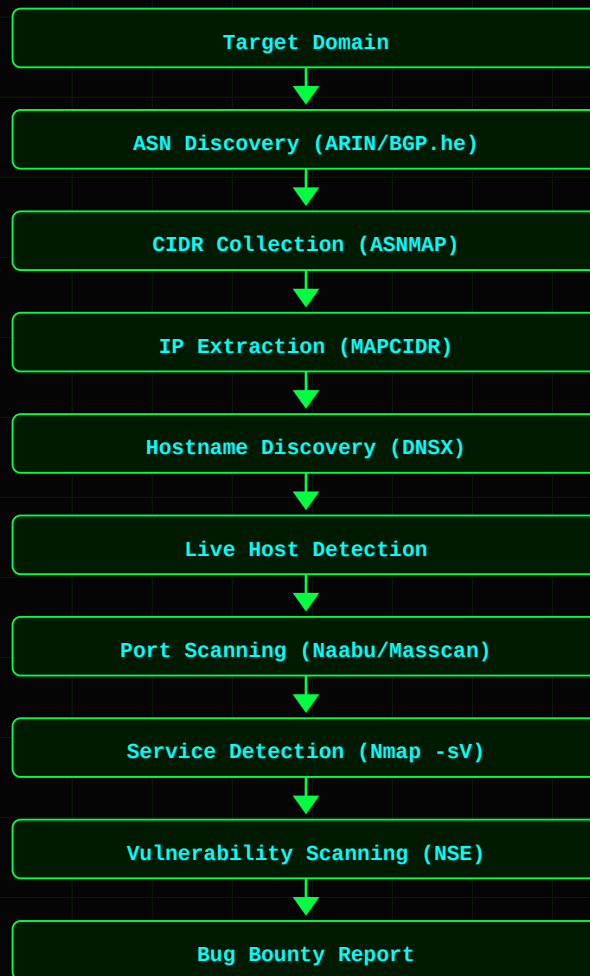
■ Phase 9: Vulnerability Discovery

Run vuln NSE scripts and check for CVEs in identified service versions.

■ Phase 10: Bug Bounty Reporting

Document findings, reproduce bugs, and create quality reports.

[COMPLETE RECON PIPELINE]



CHAPTER 1

ASN ENUMERATION

Autonomous System Numbers & CIDR Range Discovery

■ WHAT IS AN ASN?

An Autonomous System Number (ASN) is a globally unique identifier assigned to a network or group of IP addresses managed by a single organization (e.g., T-Mobile, Amazon AWS, Apple Inc.). ASNs are fundamental to internet routing and help identify which IP ranges belong to a target organization during bug bounty recon.

■ WHAT IS CIDR?

Classless Inter-Domain Routing (CIDR) notation represents a range of IP addresses. For example, 82.24.0.0/22 covers 1024 IP addresses (from 82.24.0.0 to 82.24.3.255). Each ASN owns multiple CIDR blocks that together form their IP address space.

■ DISCOVERY TOOLS & WEBSITES

■ <https://whois.arin.net/ui/>

ARIN WHOIS – Find ASN numbers for North American organizations

■ <https://bgp.he.net/>

Hurricane Electric BGP Toolkit – Global ASN & BGP routing data

■ <https://mxtoolbox.com/SuperTool.aspx>

MX Toolbox – Verify CIDR ranges and network info

■ <https://asnlookup.com/>

ASN Lookup – Quick ASN to CIDR mapping

PRO TIP

Pro Tip: Search for the company name in ARIN/BGP.he.net and collect ALL ASN numbers. Large companies often have multiple ASNs across different regions and subsidiaries.

■ TOOL: ASNMAP

■ What is this tool?

ASNMAP is a Go-based CLI tool from ProjectDiscovery that maps organization names or ASN numbers to their complete list of CIDR ranges. It queries multiple data sources including ARIN, RIPE, and APNIC to provide comprehensive IP range data.

■ Why use it?

Bug bounty hunters use ASNMAP to quickly extract all IP ranges belonging to a target without manually querying multiple WHOIS databases. It saves hours of manual work.

■ Installation

```
[bash]  
go install github.com/projectdiscovery/asnmmap/cmd/asnmmap@latest
```

■ Commands & Explanation

```
[bash]  
asnmmap -a AS996 > tmobilecidr.txt
```

-a AS996 → Target ASN number (T-Mobile = AS996)

> tmobilecidr.txt → Redirect all CIDR output to a file

```
[bash]  
asnmmap -a AS714 > apple_cidr.txt    # Apple ASN
```

-a AS714 → Apple's ASN number

> apple_cidr.txt → Save CIDR ranges to file

■ Real World Example

Running `asnmmap -a AS996` for T-Mobile returns all their CIDR ranges like 66.94.0.0/17, 208.54.0.0/16, etc. These ranges represent all IPs owned by T-Mobile.

■ Expected Output

Output looks like: 66.94.0.0/17 208.54.0.0/16 72.14.0.0/17 ...

■ Bug Bounty Use Cases

- Find hidden assets of the target in scope
- Discover unmonitored server infrastructure
- Identify IP ranges for mass scanning
- Find staging/dev environments on related IPs

BUG BOUNTY TIP

IMPORTANT: Always check the bug bounty program scope!
Not all ASN ranges may be in-scope.
Verify with program rules before scanning.
Illegal scanning out-of-scope = ban + legal trouble!

CHAPTER 2

IP ENUMERATION

Expanding CIDR Ranges into Individual IP Addresses

■ TOOL: MAPCIDR

■ What is this tool?

MAPCIDR is a ProjectDiscovery tool that expands CIDR notation into individual IP addresses. It can process single CIDR ranges, files containing multiple CIDRs, and pipe input from other tools.

■ Why use it?

After collecting CIDR ranges from ASNMAP, you need to expand them into individual IPs for targeted scanning. MAPCIDR automates this expansion instantly.

■ Installation

```
[bash]  
go install -v github.com/projectdiscovery/mapcidr/cmd/mapcidr@latest
```

■ Commands & Explanation

```
[bash]  
mapcidr -cidr tmobilecidr.txt > allipaddress.txt
```

-cidr tmobilecidr.txt → Input file containing multiple CIDR ranges

> allipaddress.txt → Save all expanded IP addresses to file

```
[bash]  
mapcidr -cidr 82.24.0.0/22 > singlecidr.txt
```

-cidr 82.24.0.0/22 → Process a single CIDR range directly

> singlecidr.txt → Save IPs from this single range

```
[bash]  
echo 82.24.0.0/22 | mapcidr -silent | dnsx -ptr -resp-only
```

echo 82.24.0.0/22 → Pipe a single CIDR

mapcidr -silent → Expand to IPs silently (no banner)

dnsx -ptr -resp-only → Do reverse DNS on each IP

■ Real World Example

A /22 CIDR like 82.24.0.0/22 expands to 1024 individual IPs. MAPCIDR handles this instantly, outputting one IP per line.

■ Expected Output

Output example for 82.24.0.0/22: 82.24.0.0 82.24.0.1 82.24.0.2 ... 82.24.3.255

■ Bug Bounty Use Cases

- Build complete IP inventory of target organization
- Feed IPs to port scanners for mass scanning
- Combine with DNSX for hostname discovery
- Create target lists for service detection

BUG BOUNTY TIP

Large ASNs may contain millions of IPs.
Use rate limiting when scanning to avoid detection.
Filter active IPs first before deep scanning.
Store results in organized files for each phase.

■ COMBINED PIPELINE

```
# Complete ASN to IP pipeline
asnmap -a AS714 > apple_cidr.txt

# Expand all CIDRs to IPs
mapcidr -cidr apple_cidr.txt > apple_ips.txt

# Count total IPs
wc -l apple_ips.txt

# Pipe directly without saving intermediate files
asnmap -a AS714 | mapcidr -silent > all_ips.txt
```

CHAPTER 3

HOST DISCOVERY

Reverse DNS & Hostname Resolution with DNSX

■ TOOL: DNSX

■ What is this tool?

DNSX is a fast and multi-purpose DNS toolkit from ProjectDiscovery. For IP-to-hostname conversion, it uses PTR (reverse DNS) records. When given an IP address, it queries DNS to find what hostnames point to that IP.

■ Why use it?

Converting IPs back to hostnames reveals actual domain names of services. This helps identify what services are running, find forgotten subdomains, and focus on relevant targets.

■ Installation

```
[bash]
go install -v github.com/projectdiscovery/dnsx/cmd/dnsx@latest
```

■ Commands & Explanation

```
[bash]
cat allipaddress.txt | dnsx -ptr -resp-only
```

`cat allipaddress.txt` → Read all IP addresses from file

`-ptr` → Perform reverse PTR DNS lookup

`-resp-only` → Show only the resolved hostname (no IP)

```
[bash]
mapcidr -cidr tmobilecidr.txt | dnsx -ptr -resp-only
```

`mapcidr -cidr` → Expand CIDR ranges inline

`| dnsx -ptr -resp-only` → Pipe directly to DNSX for hostname lookup

```
[bash]
mapcidr -cidr allipaddress.txt | dnsx -ptr -resp-only > hostnames.txt
```

`-cidr allipaddress.txt` → Input file of IPs

`> hostnames.txt` → Save discovered hostnames to file

■ Real World Example

Running DNSX on 1000 T-Mobile IPs may reveal hostnames like `gateway.tmobile.com`, `api.t-mobile.com`, `admin.internal.tmobile.com` etc.

■ Expected Output

Hostnames output: cdn.example.com api.example.com internal-dev.example.com
staging.example.com

■ Bug Bounty Use Cases

- Discover forgotten/unmonitored subdomains via reverse DNS
- Find internal naming conventions (dev, staging, admin)
- Identify CDN vs direct IP hosts
- Cross-reference discovered hostnames with bug bounty scope

BUG BOUNTY TIP

BUG BOUNTY TIP: Hostnames with "dev", "staging", "admin", "internal", "test" in them are GOLDMINES for bugs!
These environments often have weaker security controls.

CHAPTER 4

PORT SCANNING

Naabu, Masscan, Rustscan & Angry IP Scanner

■ TOOL: NAABU

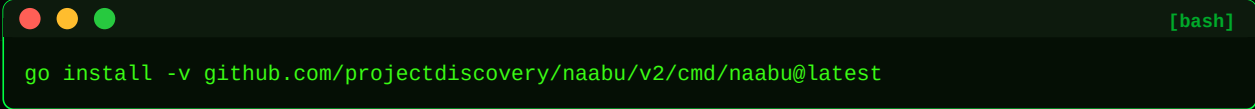
■ What is this tool?

Naabu is a fast and reliable port scanning tool from ProjectDiscovery written in Go. It supports SYN, CONNECT and UDP scan modes with rate limiting, retries, and concurrent scanning capabilities optimized for large-scale reconnaissance.

■ Why use it?

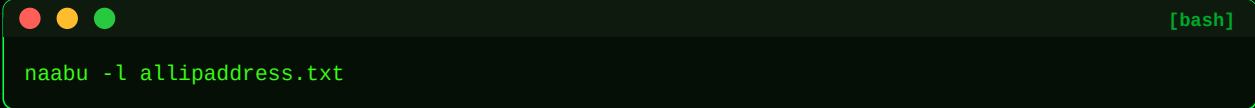
Naabu is preferred for bug bounty recon because it is fast, integrates with other ProjectDiscovery tools (like httpx, nuclei), supports massive target lists, and handles rate limiting to avoid triggering WAF/IDS.

■ Installation



```
[bash]  
go install -v github.com/projectdiscovery/naabu/v2/cmd/naabu@latest
```

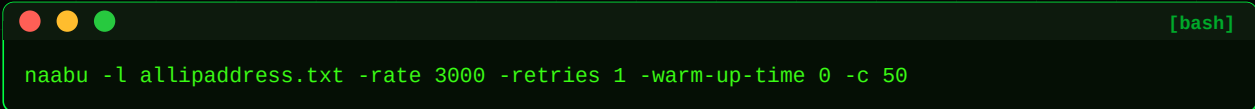
■ Commands & Explanation



```
[bash]  
naabu -l allipaddress.txt
```

-l allipaddress.txt → Load targets from file

(default) → Scans top 100 ports



```
[bash]  
naabu -l allipaddress.txt -rate 3000 -retries 1 -warm-up-time 0 -c 50
```

-l allipaddress.txt → Read target IPs from file

-rate 3000 → Send 3000 packets per second

-retries 1 → Retry failed connections once

-warm-up-time 0 → No warmup – start full speed immediately

-c 50 → Use 50 concurrent connections

■ Real World Example

Scan all T-Mobile IPs (saved in allipaddress.txt) at high speed to find open ports across thousands of IPs in minutes rather than hours.

■ Expected Output

Output example: 172.16.1.1:80 172.16.1.1:443 172.16.2.5:8080 172.16.2.9:22

■ Bug Bounty Use Cases

- Quickly scan entire ASN ranges for open ports
- Pipe output directly to httpx for web service discovery
- Find non-standard ports hosting web apps
- Identify exposed admin panels and internal services

BUG BOUNTY TIP

Pro Tip: Combine Naabu output with HTTPX:

```
naabu -l ips.txt | httpx -title -status-code
```

This gives you live web services with titles in one command!

■ TOOL: MASSCAN

■ What is this tool?

Masscan is the fastest Internet port scanner. It can scan the entire Internet in about 6 minutes by transmitting packets asynchronously using its own TCP/IP stack. It is ideal for scanning large CIDR ranges quickly.

■ Why use it?

When you need raw speed to scan thousands or millions of IPs, Masscan is the tool. It is faster than Nmap but provides less detail.

■ Installation

```
[bash]
# Install on Ubuntu/Debian ■ sudo apt install masscan ■ # Or build from source ■ git clone https://github.com/robertdavidmatt/masscan
```

■ Commands & Explanation

```
[bash]
masscan -p0-65535 66.207.176.0/22 --rate=50000
```

`-p0-65535` → Scan all 65535 ports

`66.207.176.0/22` → Target CIDR range

`--rate=50000` → Send 50,000 packets per second

```
[bash]
masscan -iL allipaddress.txt --rate=50000 --ports 0-65535 -v
```

`-iL allipaddress.txt` → Load targets from file

`--rate=50000` → Scan speed (packets/sec)

`--ports 0-65535` → Scan all ports

`-v` → Verbose output

■ Real World Example

Scanning a /22 CIDR (1024 IPs) across all 65535 ports using Masscan at 50000 pps completes in seconds to minutes depending on network conditions.

■ Expected Output

Output: Discovered open port 443/tcp on 66.207.176.5 Discovered open port 80/tcp on 66.207.176.10

■ Bug Bounty Use Cases

- Rapid discovery of open ports across large IP ranges
- First-pass scanning before detailed Nmap enumeration
- Finding services on non-standard ports quickly

BUG BOUNTY TIP

WARNING: High rate scanning can get your IP banned!
Start with --rate=1000 on bug bounty programs.
Always check program rules for rate limits.

■ TOOL: RUSTSCAN

■ What is this tool?

Rustscan is a modern port scanner written in Rust, designed to be the fastest port scanner available. It auto-pipes results to Nmap for detailed analysis, combining speed with depth.

■ Why use it?

Rustscan dramatically reduces the time needed for initial port discovery before handing off to Nmap for service detection. What takes Nmap minutes, Rustscan does in seconds.

■ Installation

```
[bash]
# Install via cargo (Rust) ■ cargo install rustscan ■ # Or Docker ■ docker pull rustscan/rustscan:2.1.1
```

■ Commands & Explanation

```
[bash]
rustscan -a allipaddress.txt -u 10000
```

-a allipaddress.txt → Target file or IP/hostname

-u 10000 → Ulimit – max open file descriptors (speed tuning)

```
[bash]
rustscan -a 192.168.1.1 --range 1-65535 -- -sV -sC
```

-a 192.168.1.1 → Single target IP

--range 1-65535 → Port range to scan

-- -sV -sC → Pass flags to Nmap after discovery

■ Real World Example

Rustscan discovers all open ports in seconds, then automatically launches Nmap on those specific ports for version detection. This is more efficient than running Nmap alone.

■ Expected Output

Rustscan finds ports → Nmap runs: PORT STATE SERVICE VERSION 80/tcp open http Apache 2.4.49 443/tcp open ssl/http nginx 1.18

■ Bug Bounty Use Cases

- Speed up initial port discovery in bug bounty
- Combine rapid discovery with Nmap's deep analysis
- Reduce total scan time by 10x compared to Nmap alone

BUG BOUNTY TIP

Rustscan Best Practice:

```
rustscan -a target.com -- -sv -sC --script vuln
```

This does: fast discovery + version + default scripts + vuln check

IMPORTANT NOTE

Angry IP Scanner: A cross-platform GUI tool available at <https://angryip.org/>. Use it for visual scanning of IP ranges. Upload your IP list, configure ports, and it shows alive hosts with open ports in an easy-to-read table interface. Great for quick visual verification of scan results.

CHAPTER 5

NMAP COMPLETE GUIDE

The Swiss Army Knife of Network Scanning

Nmap (Network Mapper) is the most powerful open-source network scanner. It discovers hosts, services, OS details, and vulnerabilities using its powerful NSE (Nmap Scripting Engine). Released in 1997, it remains the gold standard for network reconnaissance.

BASIC SCANNING

```
# Default top 1000 ports scan
nmap target.com

# Specific ports only
nmap -p 80,443 target.com

# Scan ALL 65535 ports
nmap -p- target.com

# Fast all ports (with speed)
nmap -p- --min-rate 5000 target.com

# Full featured scan (common)
nmap -sV -sC -Pn target.com -oN output.txt
```

IMPORTANT FLAGS

FLAG	PURPOSE	WHEN TO USE
-sV	Version detection	Always – identifies service versions
-O	OS detection	Identify target operating system
-A	Aggressive (all)	Full info: OS + version + scripts
-Pn	Skip ping check	When firewall blocks ICMP
-T4	Fast timing	Default for most scans
-T1	Sneaky timing	Evade IDS/WAF detection
--open	Only open ports	Filter out closed/filtered
-sC	Default NSE scripts	Run safe discovery scripts
--min-rate	Min packets/sec	Speed up scan
-oN file	Normal output	Save readable results
-oX file	XML output	Machine-parseable results
-oA all	All formats	Save all output formats

SCAN TYPES


```
[bash]

# SYN Stealth Scan (requires root) – fastest & stealthy
sudo nmap -sS target.com

# TCP Connect Scan (no root needed)
nmap -sT target.com

# UDP Scan (requires root) – finds DNS, SNMP, etc.
sudo nmap -sU target.com

# NULL Scan – sends packet with no flags
sudo nmap -sN target.com

# FIN Scan – sends FIN flag only
sudo nmap -sF target.com

# XMAS Scan – FIN, PSH, URG flags set
sudo nmap -sX target.com
```

BUG BOUNTY TIP

BUG BOUNTY TIP: Use -sS (SYN) for stealth.
SYN scans do not complete the TCP handshake,
making them harder to detect in application logs.
NULL/FIN/XMAS scans can bypass some stateless firewalls!

■ NSE SCRIPTS – NMAP SCRIPTING ENGINE

NSE scripts are Lua programs that extend Nmap with advanced capabilities including vulnerability detection, service enumeration, authentication testing, and exploit checks. NSE scripts are located at: /usr/share/nmap/scripts/

```
[bash]

# Run default safe scripts
nmap -sC target.com

# Run vulnerability check scripts
nmap --script vuln target.com

# Auth scripts (check for default passwords)
nmap --script auth target.com

# Discovery scripts
nmap --script discovery target.com

# Brute force (careful on prod!)
nmap --script brute target.com

# Exploit scripts (use only with permission!)
nmap --script exploit target.com
```

■ SERVICE-SPECIFIC NSE SCRIPTS

COMMON MISTAKE

Created By Nitish Mukhiya | github.com/nitishmukhiyaji [PAGE 18]

CHAPTER 6

WAF & FIREWALL EVASION

Stealth Scanning Techniques

WAF (Web Application Firewall) and IDS (Intrusion Detection System) can block or alert on aggressive scans. These techniques help bypass detection while gathering intelligence.

■ Technique 1: Packet Fragmentation

```
[bash]
# Fragment packets to confuse IDS/WAF
sudo nmap -f target.com      # Single fragmentation
sudo nmap -ff target.com     # Double fragmentation
# Packets split into smaller pieces – harder to reassemble and inspect
```

PRO TIP

IDS systems may fail to reassemble fragmented packets, allowing your scan to slip through undetected.

■ Technique 2: Decoy Scanning

```
[bash]
# Scan appears to come from multiple sources
sudo nmap -D RND:10 target.com      # 10 random decoy IPs
sudo nmap -D 1.2.3.4,5.6.7.8,ME target.com # Specific decoys + your IP
# Target sees traffic from many IPs – hard to identify real attacker
```

PRO TIP

Decoy scans create noise – target sees connections from many IPs making it hard to identify and block the real scanner.

■ Technique 3: Source Port Manipulation

```
[bash]
# Firewalls often allow traffic from trusted ports
nmap --source-port 53 target.com  # Appear as DNS traffic
nmap --source-port 80 target.com  # Appear as HTTP traffic
# Many firewalls allow port 53 (DNS) traffic outbound!
```

BUG BOUNTY TIP

BUG BOUNTY TIP: Source port 53 is often whitelisted in firewall rules because DNS traffic needs to pass through.

■ Technique 4: Slow Scan (IDS Evasion)

```
[bash]
# Stay below IDS detection thresholds
nmap -T1 --scan-delay 5s target.com    # 5 second delay between probes
# Very slow but almost undetectable by rate-based IDS
```

IMPORTANT NOTE

IMPORTANT: T1 scans can take hours/days on large ranges.
Use only when stealth is critical and time is not a concern.

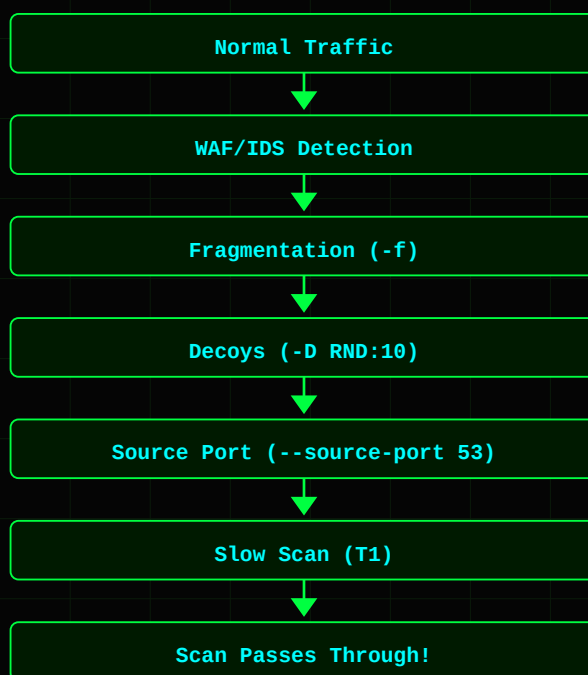
■ Technique 5: Random Host Order

```
[bash]
# Randomize scan order to avoid pattern detection
nmap --randomize-hosts -iL hosts.txt
# Sequential scanning is easier for IDS to detect and block
```

PRO TIP

Random host order makes traffic look less like a systematic scan
and more like normal distributed traffic patterns.

[WAF EVASION STRATEGY]



CHAPTER 7

BUG BOUNTY WORKFLOWS

Elite Recon Pipelines for Maximum Impact

■ Workflow 1: Quick Bug Bounty Scan

```
# Fast top 1000 ports + version detection
nmap -T4 -Pn -sV --open \
  --min-rate 5000 \
  example.com \
  -oN quick_scan.txt
```

- -T4: Fast timing template
- -Pn: Skip ping (assume host is up)
- -sV: Detect service versions
- --open: Show only open ports
- --min-rate 5000: Minimum 5000 packets/sec
- -oN: Save to normal text file

BUG BOUNTY TIP

Use for initial quick assessment. Completes in 1-3 minutes.

■ Workflow 2: Full Port Scan

```
# Scan ALL 65535 ports thoroughly
sudo nmap -T4 -Pn -p- -sV -sC \
  --min-rate 5000 \
  --max-retries 1 \
  example.com \
  -oA full_scan
```

- -p-: Scan all 65535 ports
- -sV: Version detection
- -sC: Default NSE scripts
- --max-retries 1: Retry once on failure
- -oA: Save in all formats (txt, xml, gnmap)

BUG BOUNTY TIP

Complete scan – run overnight on important targets.

■ Workflow 3: Mass Scan Multiple Hosts

```
[bash]
# Scan many hosts from a file
nmap -iL live_hosts.txt \
  -Pn \
  --min-rate 5000 \
  --max-retries 1 \
  --max-scan-delay 20ms \
  -T4 \
  --top-ports 1000 \
  --open \
  -oX nmap_mass.xml
```

- -iL live_hosts.txt: Load from file
- --max-scan-delay 20ms: Cap delay between probes
- --top-ports 1000: Scan most common 1000 ports
- -oX: XML output for parsing/importing to tools

BUG BOUNTY TIP

Use after initial host discovery to scan many targets efficiently.

■ Workflow 4: Interesting Ports Focus

```
[bash]
# Focus on high-value bug bounty ports
PORTS="21,22,23,25,53,80,110,111,135,139,143"
PORTS="$PORTS,443,445,993,995,1723,3306,3389"
PORTS="$PORTS,5900,6379,8080,8443,8888,9200"
PORTS="$PORTS,27017,28017,50000"

nmap -Pn -p $PORTS \
  -sV --open -T4 \
  example.com \
  -oN interesting_ports.txt
```

- Targets all services known to have high bug frequency
- Much faster than full port scan
- Covers databases, admin panels, remote access

BUG BOUNTY TIP

Perfect balance of speed and coverage for bug bounty work.

■ WORKFLOW 5: COMPLETE ELITE PIPELINE SCRIPT

```
[bash]

#!/bin/bash
# nmap_elite.sh - Complete automated scanning pipeline

TARGET=$1
DIR="nmap_${TARGET}"
mkdir -p $DIR

echo ">>> NMAP ELITE SCAN: $TARGET"

# Phase 1: Quick scan - top 1000 ports
echo "[*] Phase 1: Quick Scan..."
nmap -T4 -Pn --open --min-rate 5000 $TARGET -oN $DIR/quick.txt

# Phase 2: Full port scan - all 65535 ports
echo "[*] Phase 2: Full Port Scan..."
nmap -T4 -Pn -p- --open --min-rate 5000 --max-retries 1 $TARGET -oN $DIR/full_ports.txt

# Phase 3: Extract open ports from results
OPEN_PORTS=$(grep "[0-9]" $DIR/full_ports.txt | awk '{print $1}' | cut -d/ -f1 | tr '\n' ',' | sed 's/,,$//')
echo "[+] Open Ports: $OPEN_PORTS"

# Phase 4: Deep scan on open ports only
echo "[*] Phase 3: Deep Scan on Open Ports..."
sudo nmap -T4 -Pn -p $OPEN_PORTS -sV -sC --script vuln $TARGET -oA $DIR/deep_scan

# Phase 5: Report high-value services
echo "[!] High Value Services Found:"
grep -E "redis|mongodb|elasticsearch|jenkins|ftp-anon|mysql-empty|heartbleed|smb-vuln|vnc|telnet" $DIR/deep_scan

echo "[*] Results saved in: $DIR/"

# Usage: chmod +x nmap_elite.sh && ./nmap_elite.sh example.com
```

PRO TIP

ELITE TIP: This script runs in 3 phases:

1. Quick scan to see what is there
2. Full port scan to miss nothing
3. Deep scan only on confirmed open ports (saves time!)

This "funnel" approach is used by top bug bounty hunters.

CHAPTER 8

HIGH VALUE PORTS

Bug Hunter's Target Priority Guide

Not all open ports are equal. These ports historically yield the highest return for bug bounty hunters due to weak default configurations, known CVEs, and common misconfigurations in production environments.

PORT	SERVICE	RISK LEVEL	ATTACK SURFACE	BUG BOUNTY VALUE
21	FTP	■ HIGH	Anonymous login, file access	★★★★■
22	SSH	■ MEDIUM	Weak creds, old versions	★★★██
23	Telnet	■ CRITICAL	Plaintext credentials!	★★★★★
25	SMTP	■ MEDIUM	Email spoofing, open relay	★★★██
53	DNS	■ MEDIUM	Zone transfer, subdomain enum	★★★██
80	HTTP	■ STANDARD	Web vulnerabilities	★★★██
443	HTTPS	■ STANDARD	Web vulns + SSL issues	★★★██
3306	MySQL	■ HIGH	Empty password, ext access	★★★★★
5432	PostgreSQL	■ HIGH	No auth, default creds	★★★★★
6379	Redis	■ CRITICAL	No auth = full DB access!	★★★★★
8080	HTTP-Alt	■ MEDIUM	Admin panels, dev servers	★★★★■
8443	HTTPS-Alt	■ MEDIUM	Admin panels, APIs	★★★★■
9200	Elasticsearch	■ CRITICAL	No auth = all data exposed!	★★★★★
27017	MongoDB	■ CRITICAL	No auth = all data exposed!	★★★★★
5900	VNC	■ CRITICAL	No password = desktop access!	★★★★★
2375	Docker API	■ CRITICAL	No auth = root on server!	★★★★★
11211	Memcached	■ HIGH	Data exposure, DDoS amp	★★★★■
50000	Jenkins	■ CRITICAL	No auth = RCE possible!	★★★★★
28017	MongoDB Web	■ HIGH	Web UI data exposure	★★★★■

COMMON MISTAKE

CRITICAL: Ports 6379 (Redis), 9200 (Elasticsearch), 27017 (MongoDB), 5900 (VNC), 2375 (Docker API), and 50000 (Jenkins) with NO authentication are often worth \$1,000-\$10,000+ in bug bounty programs! These are P1/Critical severity findings.

CHAPTER 9

HOST DISCOVERY TECHNIQUES

Finding Live Hosts Before Port Scanning

Before port scanning, identify which hosts are actually alive. Scanning dead IPs wastes time. These techniques use different protocol layers to discover live hosts.

■ ARP Scan

Best for local network discovery. ARP works only on the same subnet.

```
nmap -PR -sn 172.21.230.1/24 --reason -vv
```

- PR → Use ARP for discovery
- sn → No port scan (ping/discovery only)
- reason → Show why host is considered up/down
- vv → Very verbose – show all activity

PRO TIP

ARP replies cannot be blocked by firewalls within the same network segment.

■ ICMP Echo Discovery

Traditional ping-based discovery. Fast but easily blocked.

```
nmap -PE -sn 172.21.230.1/24 --reason
```

- PE → Send ICMP Echo (ping) requests
- sn → Skip port scan – discovery only
- reason → Explain why each host is up/down

PRO TIP

Many modern firewalls block ICMP. Use TCP-based methods if ICMP fails.

■ TCP SYN Discovery

Sends SYN to port 80/443. If host is up, it responds with SYN-ACK or RST.

```
nmap -PS -sn 172.21.230.1/24 --reason
```

- PS → Send TCP SYN packets to port 80
- sn → Discovery only – no port scan

--reason → Show discovery reason

PRO TIP

More reliable than ICMP when firewalls block ping.

■ TCP ACK Discovery

ACK packets elicit RST responses from live hosts, revealing their presence.



[bash]

```
nmap -PA 172.21.230.1/24 --reason -sn -vv
```

-PA → Send TCP ACK packets

-sn → Skip port scan

--reason -vv → Verbose with reasons

PRO TIP

Great for bypassing stateful firewalls that block SYN but allow ACK.

■ UDP Discovery

Sends UDP to specific ports. Live hosts respond with ICMP port unreachable.



[bash]

```
nmap -PU -sn 172.21.230.1/24
```

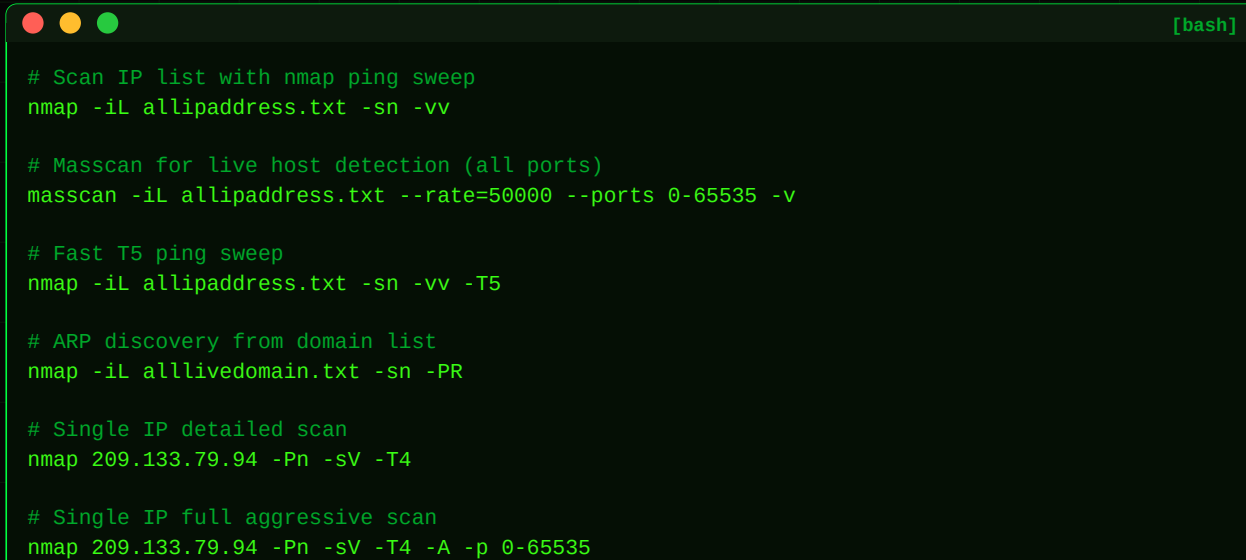
-PU → Send UDP packets for discovery

-sn → Discovery only mode

PRO TIP

Slower than TCP but finds hosts that only have UDP services running.

■ BULK HOST DISCOVERY COMMANDS



```
[bash]

# Scan IP list with nmap ping sweep
nmap -iL allipaddress.txt -sn -vv

# Masscan for live host detection (all ports)
masscan -iL allipaddress.txt --rate=50000 --ports 0-65535 -v

# Fast T5 ping sweep
nmap -iL allipaddress.txt -sn -vv -T5

# ARP discovery from domain list
nmap -iL alllivedomain.txt -sn -PR

# Single IP detailed scan
nmap 209.133.79.94 -Pn -sV -T4

# Single IP full aggressive scan
nmap 209.133.79.94 -Pn -sV -T4 -A -p 0-65535
```

CHAPTER 10

SHODAN RECON

Internet-Wide Intelligence Gathering

Shodan is a search engine for internet-connected devices. It indexes server banners, service information, open ports, and vulnerability data across the entire internet. For bug bounty hunters, Shodan reveals exposed services without active scanning.

■ SHODAN SEARCH WORKFLOW

■ Step 1

[shodan]

```
# Shodan Search Query
hostname:target.com
```

Purpose: Find all IPs associated with the target domain

■ Step 2

[shodan]

```
# Shodan Search Query
hostname:apple.com -http.title:"Invalid URL"
```

Purpose: Filter out invalid/error pages

■ Step 3

[shodan]

```
# Shodan Search Query
hostname:target.com port:8080
```

Purpose: Find specific services on non-standard ports

■ Step 4

[shodan]

```
# Shodan Search Query
hostname:target.com org:"Apple Inc"
```

Purpose: Filter by organization name

■ Step 5

[shodan]

```
# Shodan Search Query
hostname:target.com has_screenshot:true
```

Purpose: Find exposed web interfaces with screenshots

■ AFTER SHODAN – VERIFY & SCAN

```
[bash]

# Step 1: Collect IP from Shodan results
# Open terminal – run whois to identify IP ownership
whois 16.65.101.114
# Verify: Is this IP owned by target? Not AWS/CDN?

# Step 2: Version detection scan
nmap -sV 16.65.101.114 -T4 -A

# Step 3: Aggressive scan with rate limits
nmap -sV 16.65.101.114 -T4 -A --min-rate 1000 --max-retries 3 -Pn

# Step 4: Run HTTP vuln scripts on interesting IPs
nmap 16.65.101.114 -T4 --script "http-vuln*"

# Step 5: Custom NSE scripts
nmap 192.168.29.183 -sV --script whois-ip.nse
nmap 192.168.64.128 -sV --script "ftp-*" -vv

# NSE scripts location:
# /usr/share/nmap/scripts/
# Reference: https://www.stationx.net/nmap-cheat-sheet/
```

IMPORTANT NOTE

IMPORTANT: Always run whois on IPs found via Shodan!
Confirm the IP is directly owned by the target, not by
a CDN (Cloudflare, AWS, Fastly) or third party.
Scanning AWS-owned IPs when only target.com is in scope
can violate bug bounty rules!

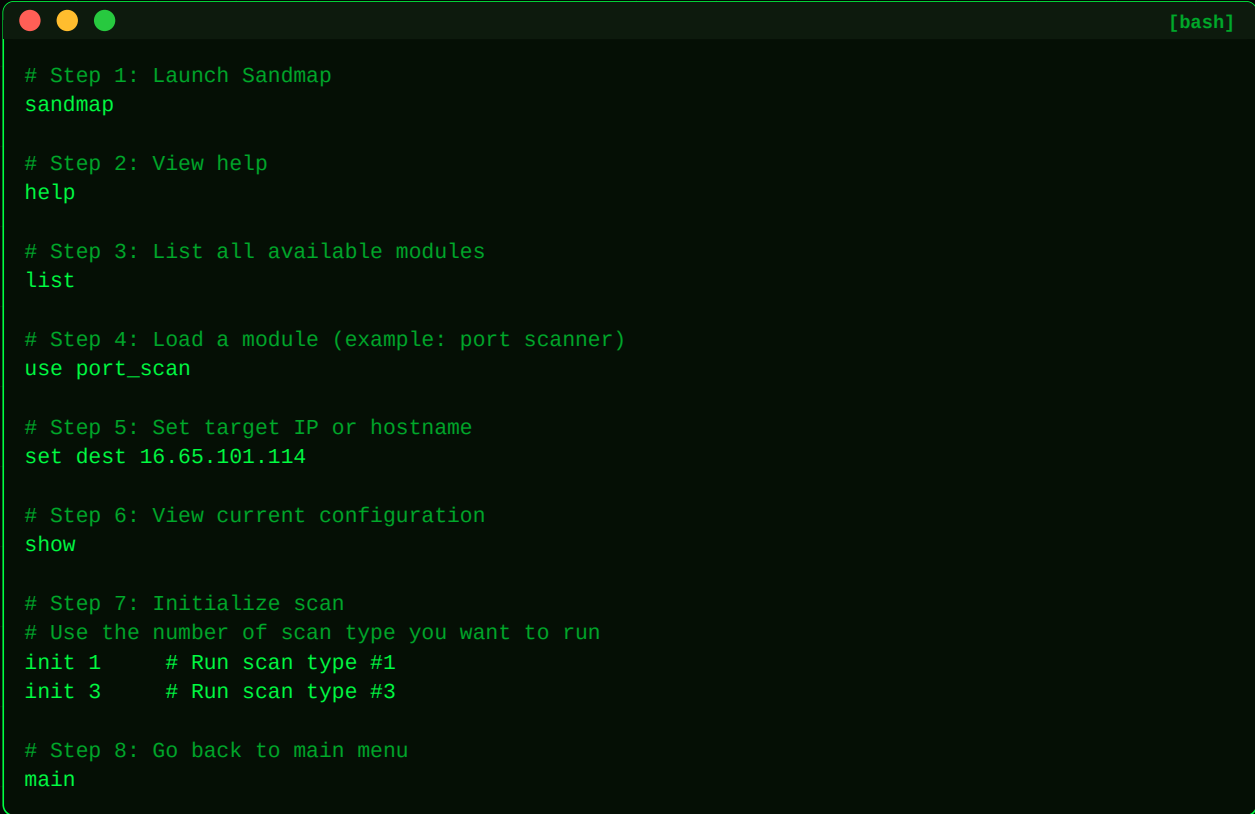
CHAPTER 11

SANDMAP

Interactive Nmap Frontend Tool

Sandmap is an interactive Nmap frontend that provides a menu-driven interface for running various scan types without memorizing complex command-line flags. It organizes scan types into modules and runs Nmap underneath.

■ SANDMAP USAGE GUIDE



```
[bash]
# Step 1: Launch Sandmap
sandmap

# Step 2: View help
help

# Step 3: List all available modules
list

# Step 4: Load a module (example: port scanner)
use port_scan

# Step 5: Set target IP or hostname
set dest 16.65.101.114

# Step 6: View current configuration
show

# Step 7: Initialize scan
# Use the number of scan type you want to run
init 1      # Run scan type #1
init 3      # Run scan type #3

# Step 8: Go back to main menu
main
```

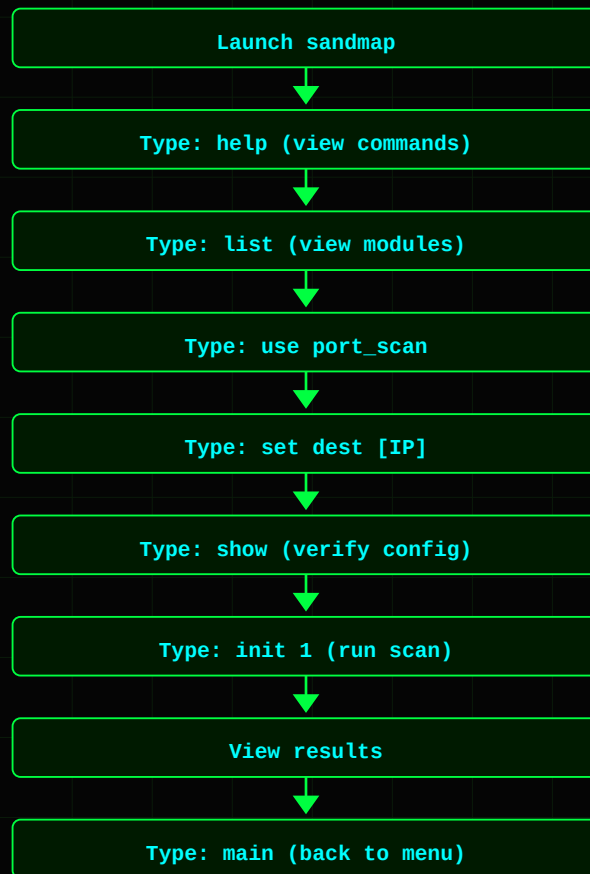
PRO TIP

SANDMAP WORKFLOW:

Launch → help → list → use [module] → set dest [ip] → show → init [number]

The "init" command takes the number of the scan type from the module list. Check the list after loading a module to see available scan types and their numbers.

[SANDMAP WORKFLOW]



CHAPTER 12

SCANCANNON

Automated CIDR Range Scanning Script

ScanCannon is a bash script wrapper that automates Masscan and Nmap scanning for CIDR ranges. It takes CIDR ranges as input and automates the process of initial discovery, port scanning, and service detection.

■ SETUP & USAGE

```
[bash]
# Step 1: Clone ScanCannon repository
git clone https://github.com/johnnyxmas/scancannon
cd scancannon

# Step 2: Make executable
chmod +x scancannon.sh

# Step 3: Create CIDR input file
# Navigate to the scancannon directory
# Edit cidr.txt with your target CIDR ranges
nano cidr.txt

# Add CIDR ranges (one per line):
# 192.168.1.0/24
# 10.0.0.0/8
# 172.16.0.0/12

# Step 4: Run ScanCannon
./scancannon.sh cidr.txt

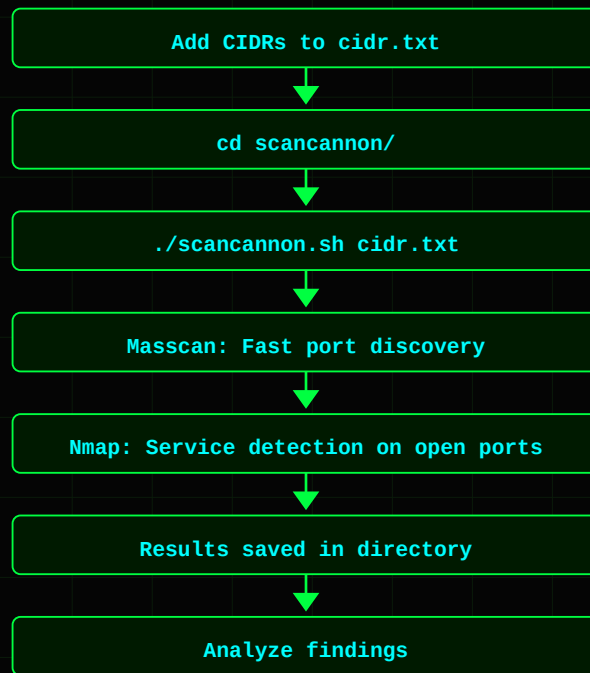
# IMPORTANT: Run from inside the scancannon directory!
# The script looks for cidr.txt in its own directory
```

IMPORTANT NOTE

SCANCANNON KEY NOTES:

- Navigate to the scancannon directory before running
- Put your CIDR ranges in cidr.txt in that same directory
- The script will run masscan then nmap automatically
- Results are saved in the scancannon directory
- Requires root/sudo for some scan types

[SCANCANNON PIPELINE]



CHAPTER 13

COMMAND CHEAT SHEET

Quick Reference – All Commands By Tool

■ ASNMAP

Find ASN CIDR ranges	<code>asnmap -a AS996 > cidr.txt</code>
Multiple ASNs	<code>asnmap -a AS996,AS714 > multi_cidr.txt</code>

■ MAPCIDR

Expand CIDR file to IPs	<code>mapcidr -cidr cidr.txt > ips.txt</code>
Single CIDR expansion	<code>mapcidr -cidr 82.24.0.0/22 > ips.txt</code>
Pipe CIDR expansion	<code>echo 82.24.0.0/22 mapcidr -silent</code>

■ DNSX

IP to hostname (PTR)	<code>cat ips.txt dnsx -ptr -resp-only</code>
CIDR to hostnames	<code>mapcidr -cidr cidr.txt dnsx -ptr -resp-only</code>
Single CIDR to hostnames	<code>echo 82.24.0.0/22 mapcidr -silent dnsx -ptr -resp-only</code>

■ NAABU

Scan IP list	<code>naabu -l ips.txt</code>
Fast scan with rate limit	<code>naabu -l ips.txt -rate 3000 -retries 1 -c 50</code>

■ MASSCAN

Scan CIDR all ports	<code>masscan -p0-65535 192.168.1.0/24 --rate=50000</code>
Scan from file	<code>masscan -iL ips.txt --rate=50000 --ports 0-65535</code>

■ RUSTSCAN

Scan from file	<code>rustscan -a ips.txt -u 10000</code>
Scan + pass to nmap	<code>rustscan -a target.com --range 1-65535 -- -sV -sC</code>

■ NMAP – BASIC

Default 1000 ports	<code>nmap target.com</code>
Specific ports	<code>nmap -p 80,443 target.com</code>
All 65535 ports	<code>nmap -p- target.com</code>
Fast all ports	<code>nmap -p- --min-rate 5000 target.com</code>
Version + scripts	<code>nmap -sV -sC -Pn target.com -oN out.txt</code>

Aggressive all info

nmap -A target.com

■ NMAP — SCAN TYPES

SYN Stealth	sudo nmap -sS target.com
TCP Connect	nmap -sT target.com
UDP Scan	sudo nmap -sU target.com
NULL Scan	sudo nmap -sN target.com
FIN Scan	sudo nmap -sF target.com
XMAS Scan	sudo nmap -sX target.com

■ NMAP — NSE SCRIPTS

Vuln check	nmap --script vuln target.com
HTTP vulnerability scan	nmap --script http-vuln* target.com
Anonymous FTP	nmap -p 21 --script ftp-anon target.com
Redis no-auth	nmap -p 6379 --script redis-info target.com
MySQL empty pw	nmap -p 3306 --script mysql-empty-password target.com
EternalBlue SMB	nmap --script smb-vuln-ms17-010 target.com
Heartbleed SSL	nmap --script ssl-heartbleed target.com

■ NMAP — EVASION

Fragmentation	sudo nmap -f target.com
Double fragmentation	sudo nmap -ff target.com
Decoys (10 random)	sudo nmap -D RND:10 target.com
Source port DNS	nmap --source-port 53 target.com
Slow stealth scan	nmap -T1 --scan-delay 5s target.com
Random host order	nmap --randomize-hosts -iL hosts.txt

■ HOST DISCOVERY

ARP scan	nmap -PR -sn 172.21.230.1/24 --reason -vv
ICMP ping discovery	nmap -PE -sn 172.21.230.1/24 --reason
TCP SYN discovery	nmap -PS -sn 172.21.230.1/24 --reason
TCP ACK discovery	nmap -PA 172.21.230.1/24 --reason -sn -vv
UDP discovery	nmap -PU -sn 172.21.230.1/24
Ping sweep from file	nmap -iL ips.txt -sn -vv

CHAPTER 14

MASTER MIND MAP

Network Scanning Complete Overview

NETWORK SCANNING MASTER MAP

CORE: NETWORK SCANNING MASTERCLASS

■ ■ 1. RECONNAISSANCE

- ■ ■ ASN Enumeration (ARIN, BGP.he.net)
- ■ ■ CIDR Collection (ASNMAP)
- ■ ■ IP Extraction (MAPCIDR)
- ■ ■ Shodan Intelligence Gathering

■ ■ 2. DISCOVERY

- ■ ■ Hostname Discovery (DNSX -ptr)
- ■ ■ ARP Host Discovery
- ■ ■ ICMP Echo Discovery
- ■ ■ TCP SYN/ACK Discovery
- ■ ■ UDP Host Discovery

■ ■ 3. PORT SCANNING

- ■ ■ Naabu (Fast, ProjectDiscovery)
- ■ ■ Masscan (Fastest raw speed)
- ■ ■ Rustscan (Rust-powered speed)
- ■ ■ Angry IP Scanner (GUI tool)

■ ■ 4. SERVICE DETECTION

- ■ ■ Nmap -sV (Version detection)
- ■ ■ Nmap -sC (Default scripts)
- ■ ■ Nmap -A (All info aggressive)
- ■ ■ NSE Scripts (http/ftp/smb/ssl)

■ ■ 5. VULNERABILITY SCANNING

- ■ ■ Nmap --script vuln
- ■ ■ NSE: ssl-heartbleed
- ■ ■ NSE: smb-vuln-ms17-010
- ■ ■ NSE: redis-info / mysql-empty-password

■ ■ 6. EVASION

- ■ ■ Fragmentation (nmap -f)
- ■ ■ Decoys (nmap -D RND:10)
- ■ ■ Source Port (--source-port 53)
- ■ ■ Slow Scan (T1 --scan-delay 5s)

■ ■ 7. BUG BOUNTY WORKFLOW

- ■ Quick Scan → Full Scan → Deep Scan
- ■ Elite Pipeline Script
- ■ High Value Port Hunting

TOOL RELATIONSHIPS



[pipeline]

```
ASN Numbers → [ASNMAP] → CIDR Ranges
CIDR Ranges → [MAPCIDR] → IP Addresses
IP Addresses → [DNSX -ptr] → Hostnames
IP Addresses → [Naabu/Masscan/Rustscan] → Open Ports
Open Ports → [Nmap -sV] → Services & Versions
Services → [Nmap NSE Scripts] → Vulnerabilities
Vulnerabilities → Bug Bounty Report → $$$
```

**Created By Nitish Mukhiya**

linkedin.com/in/nitishmukhiya | x.com/NITISHMUKHIYAJ |
github.com/nitishmukhiyaji

This handbook is for educational purposes and authorized security testing only.

Always get written permission before scanning any system you do not own.